CITY CARE

Documentación completa 3.1 Módulo de reportes

Base de datos, backend, validaciones, evidencia y pruebas

La etapa 3.1 incorpora la recepción autenticada de reportes ciudadanos, su persistencia transaccional en MySQL y la preparación privada de la fotografía. El módulo queda listo para sustituir el almacenamiento temporal por MinIO durante la etapa 3.2 sin cambiar los datos principales del reporte.

Indicador	Definición
Estado	Completo
Endpoint nuevo	POST /api/v1/reports
Tablas nuevas	3
Categorías iniciales	9
Migraciones totales	10 completadas en MySQL
Suite funcional	52 de 52 pruebas aprobadas

Resultado

MySQL contiene las tablas report_categories, reports y report_evidences. Cada tabla tiene una llave primaria id autoincremental y las relaciones están protegidas mediante llaves foráneas. El catálogo inicial quedó insertado con IDs del 1 al 9.

1 Alcance funcional

- Crear un reporte únicamente para un usuario autenticado.
- Recibir datos y fotografía en una petición multipart/form-data.
- Validar categoría, descripción, ubicación, origen GPS o manual y evidencia.
- Generar el folio público y el estado inicial en el servidor.
- Guardar el reporte y la metadata de evidencia dentro de una transacción.
- Conservar la fotografía en un espacio privado de preparación hasta integrar MinIO.
- Publicar la operación en OpenAPI y regenerar el cliente Orval.
- Cubrir el comportamiento principal y los rechazos mediante pruebas funcionales.

2 Arquitectura del flujo

Paso	Componente	Responsabilidad
1	Aplicación móvil	Forma el multipart con texto, coordenadas y fotografía.
2	Middleware auth	Valida el token y aporta el usuario real.
3	Validador VineJS	Comprueba tipos, límites, rangos y archivo.
4	ReportsController	Aplica reglas de negocio y coordina la operación.
5	Staging service	Mueve la imagen a almacenamiento privado con nombre UUID.
6	Transacción Lucid	Crea reports y report_evidences de manera atómica.
7	MySQL	Conserva los datos, relaciones, folio y metadata.
8	Respuesta HTTP	Devuelve 201 y la representación pública del reporte.

3 Decisiones de diseño

El usuario no se acepta como dato del formulario porque podría falsificarse. Se obtiene exclusivamente desde el token. El frontend envía el nombre visible de la categoría y el backend resuelve su ID, actividad y dependencia. La fotografía no se guarda como BLOB en MySQL; solo se registra su metadata y la clave del objeto.

4 Modelo de datos

4.1 Tabla report_categories

Columna	Tipo	Regla	Uso
id	INT UNSIGNED	PK autoincremental	Identificador interno.
dependency_id	INT UNSIGNED	FK nullable a dependencies.id; RESTRICT	Dependencia que atenderá la categoría.
code	VARCHAR 50	Único y obligatorio	Código estable para integraciones.
name	VARCHAR 100	Único y obligatorio	Nombre que envía el frontend.
description	TEXT	Nullable	Descripción administrativa.
requires_description	BOOLEAN	Default false	Activa descripción obligatoria.
is_active	BOOLEAN	Default true	Permite baja lógica de la categoría.
created_at	TIMESTAMP	Obligatorio	Fecha de creación.
updated_at	TIMESTAMP	Obligatorio	Fecha de actualización.

4.2 Tabla reports

Columna	Tipo	Regla	Uso
id	INT UNSIGNED	PK autoincremental	Identificador del reporte.
folio	VARCHAR 32	Único y obligatorio	Folio público CC-AÑO-ID.
user_id	INT UNSIGNED	FK users.id; RESTRICT	Autor autenticado.
category_id	INT UNSIGNED	FK report_categories.id; RESTRICT	Clasificación elegida.
dependency_id	INT UNSIGNED	FK dependencies.id; nullable; RESTRICT	Dependencia asignada por categoría.
description	TEXT	Nullable	Detalle ciudadano.
address	VARCHAR 500	Nullable	Dirección legible detectada.
latitude	DECIMAL 9,6	Obligatorio	Latitud geográfica.
longitude	DECIMAL 9,6	Obligatorio	Longitud geográfica.
location_accuracy	DECIMAL 10,2	Nullable	Precisión aproximada en metros.
location_source	VARCHAR 20	Obligatorio	gps o manual.
status	VARCHAR 20	Default pending	Estado interno inicial.
created_at	TIMESTAMP	Obligatorio	Fecha de creación.
updated_at	TIMESTAMP	Obligatorio	Fecha de actualización.

4.3 Tabla report_evidences

Columna	Tipo	Regla	Uso
id	INT UNSIGNED	PK autoincremental	Identificador de evidencia.
report_id	INT UNSIGNED	FK reports.id; CASCADE	Reporte propietario.
kind	VARCHAR 20	Default image	Tipo de evidencia.
storage_provider	VARCHAR 30	Default local_staging	Proveedor actual.
storage_status	VARCHAR 20	Default staged	Estado de transferencia.
bucket	VARCHAR 100	Nullable	Bucket futuro de MinIO.
object_key	VARCHAR 500	Único y obligatorio	Ruta lógica del archivo.
original_name	VARCHAR 255	Obligatorio	Nombre recibido del cliente.
mime_type	VARCHAR 100	Obligatorio	Tipo MIME validado.
size_bytes	BIGINT UNSIGNED	Obligatorio	Tamaño del archivo.
created_at	TIMESTAMP	Obligatorio	Fecha de creación.
updated_at	TIMESTAMP	Obligatorio	Fecha de actualización.

4.4 Relaciones verificadas en MySQL

Origen	Columna	Destino	Borrado
report_categories	dependency_id	dependencies.id	RESTRICT
reports	user_id	users.id	RESTRICT
reports	category_id	report_categories.id	RESTRICT
reports	dependency_id	dependencies.id	RESTRICT
report_evidences	report_id	reports.id	CASCADE

4.5 Índices

Se añadieron índices para categoría activa, dependencia, usuario y fecha, estado, coordenadas, reporte de evidencia y estado de almacenamiento. folio, code, name y object_key cuentan además con unicidad.

5 Catálogo real cargado en MySQL

La consulta directa realizada después de ejecutar el seeder confirmó los siguientes registros. Todos están activos y dependency_id permanece en null hasta definir el enrutamiento administrativo de cada categoría.

ID	Código	Nombre	Requiere descripción	Activo
1	pothole	Bache	No	Sí
2	public_lighting	Alumbrado público	No	Sí
3	water_leak	Fuga de agua	No	Sí
4	accumulated_garbage	Basura acumulada	No	Sí
5	damaged_traffic_light	Semáforo dañado	No	Sí
6	damaged_sidewalk	Banqueta dañada	No	Sí
7	fallen_tree	Árbol caído	No	Sí
8	graffiti_or_vandalism	Grafiti o vandalismo	No	Sí
9	other	Otro	Sí	Sí

Los IDs son autoincrementales. El seeder usa updateOrCreate por code, por lo que puede ejecutarse otra vez sin duplicar las categorías existentes.

6 Migraciones

Orden	Migración	Resultado
8	create_report_categories_table	Tabla de catálogo creada.
9	create_reports_table	Tabla principal y relaciones creadas.
10	create_report_evidences_table	Tabla de metadata y relación CASCADE creada.

7 Componentes del backend

Componente	Responsabilidad
reports_controller.ts	Autenticación, reglas de negocio, transacción, folio, carga de relaciones y respuesta 201.
report.ts validator	Esquema de entrada multipart y límites de todos los campos.
report_evidence_staging_service.ts	Nombre UUID, organización por año y mes, movimiento y eliminación compensatoria.
report.ts model	Relaciones con usuario, categoría, dependencia y evidencias.
report_category.ts model	Catálogo, actividad, dependencia y regla de descripción.
report_evidence.ts model	Metadata y estado de almacenamiento.
handler.ts	Conversión de errores del módulo al formato JSON uniforme.
routes.ts	Registro de POST /api/v1/reports con middleware auth.

8 Reglas de negocio

- El usuario siempre se obtiene del token de acceso.
- Solo una categoría activa y con nombre exacto puede utilizarse.
- Otro requiere una descripción no vacía.
- Latitud debe estar entre -90 y 90; longitud entre -180 y 180.
- La combinación 0,0 se considera ubicación inválida.
- El origen solo admite gps o manual.
- La fotografía es obligatoria, debe ser JPG, JPEG o PNG y no superar 10 MB.
- El reporte nace con estado interno pending y se presenta como Pendiente.
- El folio se genera como CC-AÑO-ID con seis posiciones para el ID.

9 Atomicidad y archivos huérfanos

El reporte y la evidencia se insertan dentro de la misma transacción. Antes de abrirla se prepara la imagen para evitar mantener una conexión ocupada durante el movimiento del archivo. Si MySQL rechaza la operación, el bloque de compensación elimina la imagen preparada. Así no queda un reporte sin metadata ni un archivo sin reporte.

10 Almacenamiento de evidencia en 3.1

Elemento	Valor actual
Directorio privado	storage/report-evidence-staging
Estructura	reports/AÑO/MES/UUID.extensión
Proveedor registrado	local_staging
Estado registrado	staged
Bucket	null
Exposición pública	Ninguna
fotoUrl	No se devuelve durante 3.1

La carpeta está excluida del control de versiones. Este mecanismo es una transición segura para desarrollo, no el almacenamiento definitivo. En 3.2 se transferirá la evidencia a MinIO y se completarán bucket, provider, status y la URL de consulta.

11 Contrato OpenAPI y Orval

- El contrato documenta multipart/form-data y marca foto como binary.
- La operación tiene operationId createReport.
- La seguridad Bearer se declara en la ruta.
- Las respuestas 201, 401 y 422 están tipadas.
- Orval genera createReport y construye FormData para el frontend.
- La generación finalizó sin errores de validación del contrato.

12 Manejo de errores

Código	Significado
E_VALIDATION_ERROR	Entrada incompatible con el contrato.
E_REPORT_CATEGORY_INVALID	Categoría inexistente o inactiva.
E_REPORT_DESCRIPTION_REQUIRED	Falta descripción para Otro.
E_REPORT_LOCATION_INVALID	Ubicación 0,0.
E_REPORT_LOCATION_SOURCE_INVALID	Origen distinto de gps o manual.

13 Pruebas y validación técnica

Validación	Resultado
Creación sin autenticación	Rechazada con 401.
Creación correcta	Reporte, folio, coordenadas y evidencia verificados.
Coordenadas 0,0	Rechazadas con error de ubicación.
Categoría Otro	Descripción requerida.
Categoría desconocida o inactiva	Rechazada.
Rangos, origen y fotografía	Validaciones comprobadas.
Suite completa	52 de 52 pruebas aprobadas.
TypeScript	Sin errores.
ESLint	Sin errores.
Build	Compilación de producción correcta.
OpenAPI y Orval	Contrato válido y SDK generado.
MySQL	10 migraciones completadas y catálogo verificado.

14 Operación y mantenimiento

```
node ace migration:status
node ace migration:run
node ace db:seed --files=database/seeders/report_category_seeder.ts
npm run typecheck
npm run lint
npm test
npm run build
npm run openapi:generate
```

El seeder es idempotente por código. Antes de cambiar nombres visibles debe considerarse que el frontend de 3.1 envía el nombre de la categoría. Como mejora posterior puede exponerse un endpoint de catálogo y enviar el código estable en lugar del nombre.

15 Pendiente para 3.2

- Configurar cliente, endpoint, credenciales y bucket de MinIO mediante variables de entorno.
- Subir la evidencia con una clave única y tipo MIME correcto.
- Actualizar provider, bucket y storage_status después de una transferencia exitosa.
- Generar una URL firmada o una ruta controlada para consultar la imagen.
- Definir reintentos y recuperación para transferencias fallidas.
- Migrar o procesar los archivos que permanezcan en local_staging.
- Añadir pruebas de integración de MinIO y expiración de URLs.